Machine Learning

Classification by Logistic Regression

Hoda Eldardiry

Lecture Objectives

- Classification with probabilities
- Logistic Regression
- Convex optimization for classification

Logistic Regression

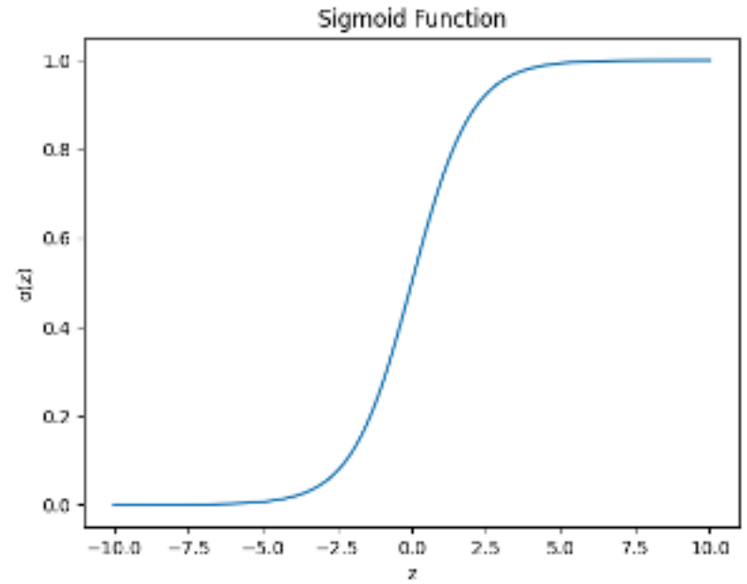
- Probabilistic Linear classifier
- Popular because: simple, powerful, flexible
- Convex optimization
- Foundation for neural networks

From Regression to Classification

- Linear regression: $\hat{y} = w^T x$
  - Works when y is continuous: $y \in R$
- But classification requires: $y \in \{0,1\}$
- Need probabilities $P(y=1|x)$
- A model that:
  - Outputs values between 0 and 1
  - Is smooth, differentiable, easy to optimize

Logistic Sigmoid Function

- Logistic Sigmoid: $\sigma(z)=1 / (1+e^{-z})$
- Properties:
  - Converts linear output into probability
  - Maps $(-\infty, \infty) \rightarrow (0,1)$
  - Smooth, differentiable



Regression for Classification?

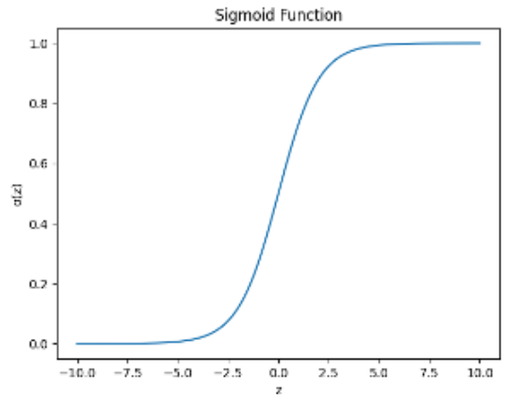
- Linear regression:
  - Fit the best linear function to the data
  - (Estimate the linear function parameters)
- Logistic regression:
  - Fit the best logistic sigmoid function to the data
  - (Estimate the logistic sigmoid parameters)

Logistic Regression Model

- Logistic Sigmoid: $\sigma(z)=1 / (1+e^{-z})$
- Logistic Regression:
  - $P(y=1|x) = \sigma(w^T x)$
  - $P(y=0|x) = 1 - \sigma(w^T x)$
- Interpretation:
  - Linear model (linear parameters)
  - Passed through nonlinear squashing (nonlinear output)
  - Produces probabilities

Decision Boundary

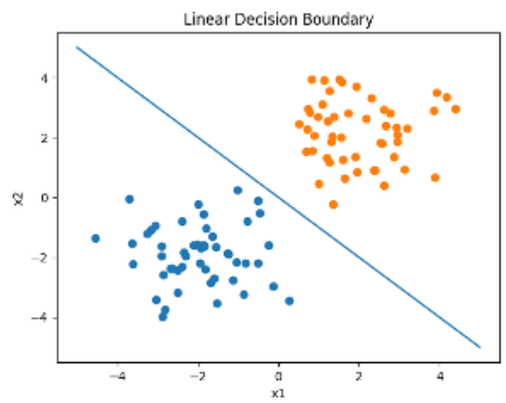
- Classification rule: predict 1 if $P(y = 1 \mid x) > 0.5$



- Since: $\sigma(w^T x) > 0.5 \iff w^T x > 0$

- Linear decision boundary: $w^T x = 0$

- *Logistic Regression is a linear classifier*



Likelihood

- Logistic Regression:
  - $P(y=1|x) = \sigma(w^T x)$
  - $P(y=0|x) = 1 - \sigma(w^T x)$
- Likelihood for one data point (x, y):
  - $P(y|x; w) = \sigma(w^T x)^y (1 - \sigma(w^T x))^{1-y}$
  - Works for: y = 0 & y = 1
- Likelihood for dataset (**X**, y): $L(w) = \prod_{i=1}^{n} P(y_i|x_i; w)$
- Goal: maximize this likelihood

Likelihood

- Likelihood for one data point (x, y):
  - $P(y|x; w) = \sigma(w^T x)^y \, (1 - \sigma(w^T x))^{1-y}$
- Likelihood for dataset (**X**, y): $L(w) = \prod_{i=1}^{n} P(y_i|x_i; w)$
- Goal: estimate ***w*** to maximize this likelihood

$$\boldsymbol{w}^* = argmax_{\boldsymbol{w}} \, \sigma(\boldsymbol{w}^T \overline{\boldsymbol{x}})^y \left(1 - \sigma(\boldsymbol{w}^T \overline{\boldsymbol{x}})\right)^{1-y}$$

Log-Likelihood & Cross-Entropy

- Goal: find ***w*** to *maximize likelihood*

$$\boldsymbol{w}^* = argmax_{\boldsymbol{w}}\, \sigma(\boldsymbol{w}^T\overline{\boldsymbol{x}})^y \left(1 - \sigma(\boldsymbol{w}^T\overline{\boldsymbol{x}})\right)^{1-y}$$

- Equivalent to: *minimizing Negative Log-Likelihood*
  - Take log: turn product into sum: easier to solve
  - Negative: turn maximization to minimization to treat as loss

$$L(\boldsymbol{w}) = -\sum_n y_n \ln \sigma(\boldsymbol{w}^T\overline{\boldsymbol{x}}_n) + (1 - y_n) \ln(1 - \sigma(\boldsymbol{w}^T\overline{\boldsymbol{x}}_n))$$

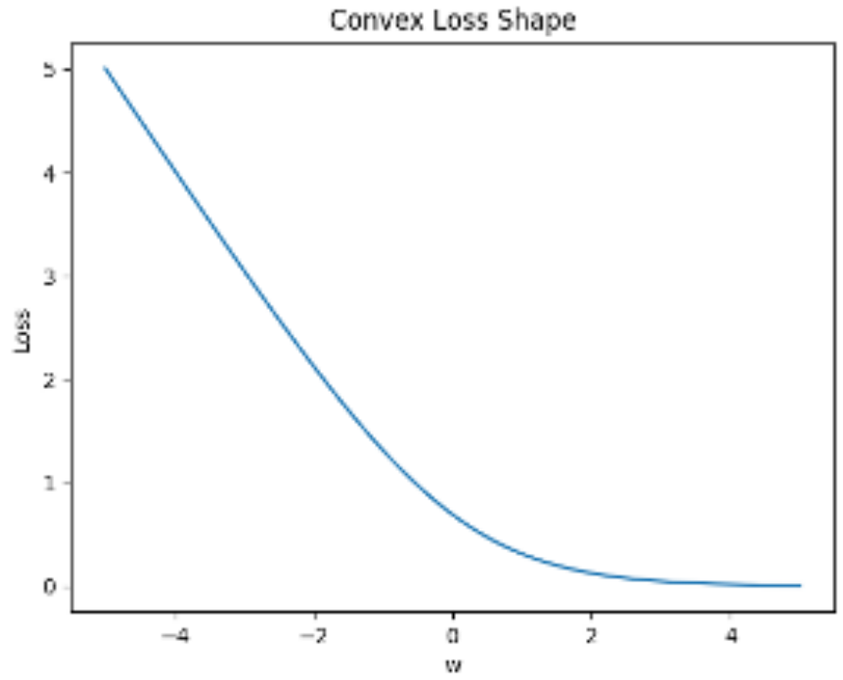
- Also called: *Binary Cross-Entropy Loss*

Why Not Squared Error?

- If we used squared loss:
  - No probabilistic interpretation
  - Not convex for logistic model
  - Can produce poor boundaries
- Cross-entropy loss:
  - Comes from maximum likelihood
  - Proper probabilistic meaning
  - Convex objective

Convexity

- Negative log-likelihood is **convex**
  - Single global minimum
  - No local minima
  - Reliable optimization



- *Convex objective ensures global optimum*

Logistic Regression

Objective: Negative Log Likelihood (Minimization)

$$L(\boldsymbol{w}) = -\sum_n y_n \ln \sigma(\boldsymbol{w}^T \bar{\boldsymbol{x}}_n) + (1 - y_n) \ln(1 - \sigma(\boldsymbol{w}^T \bar{\boldsymbol{x}}_n))$$

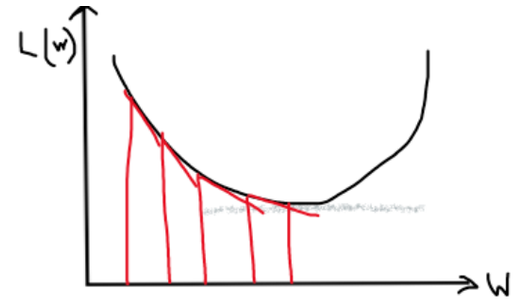
- Solve optimization to find best w that minimizes this objective

- Compute derivative, set to zero

Gradient Descent of the Loss

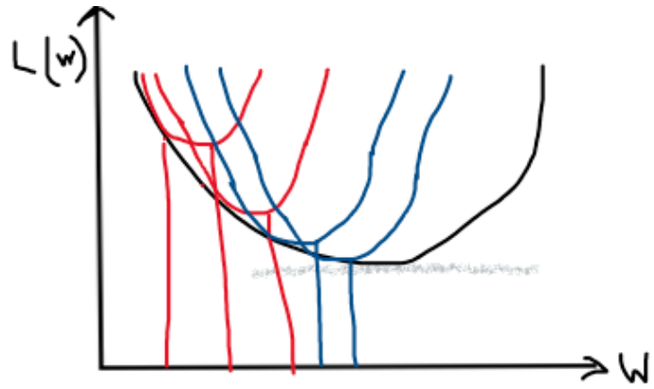
- Gradient = prediction error × input
- $\nabla L(w) = 0$

$$0 = \sum_n [\sigma(\boldsymbol{w}^T \bar{\boldsymbol{x}}_n) - y_n] \bar{\boldsymbol{x}}_n$$



- Update rule: $w := w - \eta \nabla L(w)$
- Each update pushes weights in direction of: error x input
- Pros: simple & scalable
- Cons: requires learning rate (step size) & many iterations
- Consider a more efficient method?

Newton's Method (Second-Order)



- Second-order: uses curvature (second derivatives)
- Update rule: $w := w - H^{-1} \nabla L(w)$
- Where H (Hessian matrix):

$$H = \begin{bmatrix} \frac{\partial L}{\partial^2 w_0} & \cdots & \frac{\partial L}{\partial w_0 \partial w_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial L}{\partial w_m \partial w_0} & \cdots & \frac{\partial L}{(\partial w_m)^2} \end{bmatrix}$$

Derivative w.r.t w_0, then w_m

- Pros: Faster convergence, uses curvature, no step size needed
- Cons: Hessian is expensive & not scalable to high dimensions
- Good for: Low-dimensional problems

Hessian Computation at Each Update Step

H is defined as second order derivative

$$\begin{aligned}\boldsymbol{H} &= \nabla(\nabla L(\boldsymbol{w})) \\ &= \sum_{n=1}^{N} \sigma(\boldsymbol{w}^T \bar{\boldsymbol{x}}_n)\big(1 - \sigma(\boldsymbol{w}^T \bar{\boldsymbol{x}}_n)\big)\bar{\boldsymbol{x}}_n \bar{\boldsymbol{x}}_n^T \\ &= \bar{\boldsymbol{X}} \boldsymbol{R} \bar{\boldsymbol{X}}^T\end{aligned}$$

H can be expressed in this form:

$\bar{\boldsymbol{X}}$: data with a row of 1's on top of the data

R : diagonal matrix of this form

$$\text{where } \boldsymbol{R} = \begin{bmatrix} \sigma_1(1-\sigma_1) & & \\ & \ddots & \\ & & \sigma_N(1-\sigma_N) \end{bmatrix}$$

and sigmoids are computed in this form:

$$\text{and } \sigma_1 = \sigma(\boldsymbol{w}^T \bar{\boldsymbol{x}}_1), \quad \sigma_N = \sigma(\boldsymbol{w}^T \bar{\boldsymbol{x}}_N)$$

Gradient vs Newton

Method	Per Step Cost	Iterations
Gradient Descent	Low	Many
Newton’s Method	High	Few

What Happens If Data Is Separable?

- If data is linearly separable: $|w| \to \infty$
  - Model pushes probabilities to 1
    - Problems: overfitting, numerical instability
- Problem: when minimizing loss, weights can get too big
- Solution: regularization
  - Add penalty: $\text{Loss} = \nabla L + \lambda ||w||^2$

Numerical Issues

Global optimum might fit the data too well

- Logistic Regression is subject to overfitting
  - Without enough data, logistic regression can classify each data point arbitrarily well (i.e., $\Pr(correct\ class) \rightarrow 1$)
- Problems: $weights \rightarrow \pm\infty$

  $Hessian \rightarrow singular$

=> make probability of correct class as close as possible to 1,
=> as far as possible along x-axis,
=> w large

To reach $\sigma(\boldsymbol{w}^T\bar{\boldsymbol{x}})$ —> 1

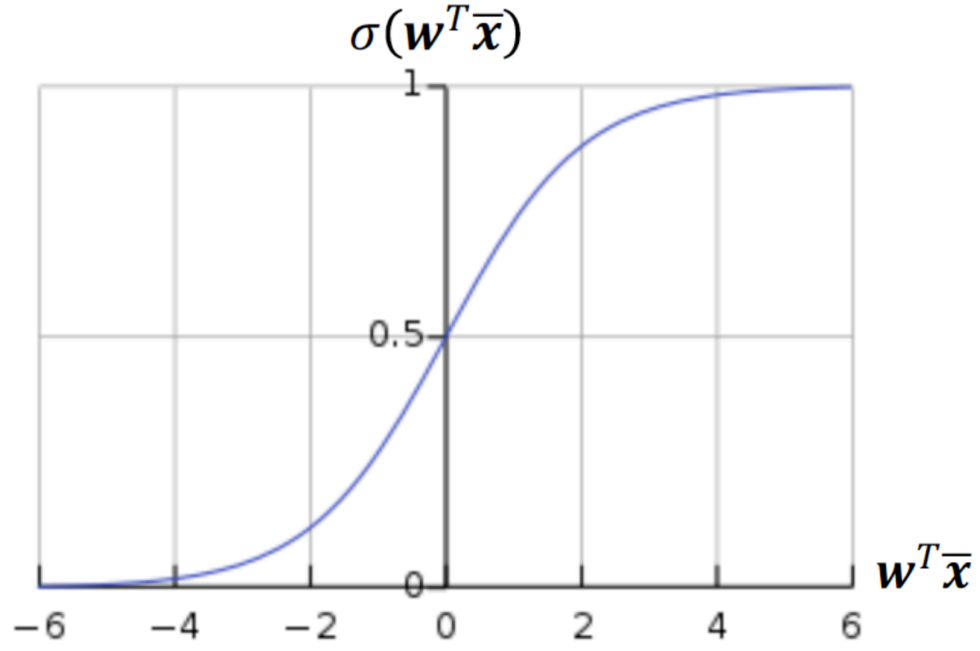
We need $\boldsymbol{w}^T\bar{\boldsymbol{x}}$ —> ∞

Therefore |w| —> ∞

R : $\begin{bmatrix} \sigma_1(1-\sigma_1) & & \\ & \ddots & \\ & & \sigma_N(1-\sigma_N) \end{bmatrix}$ —> 0

H : $\bar{\boldsymbol{X}}\boldsymbol{R}\bar{\boldsymbol{X}}^T$ —> singular

=> can't take the inverse if matrix is singular
=> can't apply Newton's method



Regularization

Solution: penalize large weights

Challenge: when minimizing loss, weights might get too large
Solution: add penalty term to minimize magnitude of weights

Objective: $\min_{\mathbf{w}} L(\boldsymbol{w}) + \frac{1}{2}\lambda\left|\left|\boldsymbol{w}\right|\right|_2^2$

$$= \min_{\boldsymbol{w}} -\sum_n y_n \ln \sigma(\boldsymbol{w}^T\bar{\boldsymbol{x}}_n) + (1-y_n)\ln(1-\sigma(\boldsymbol{w}^T\bar{\boldsymbol{x}}_n)) + \frac{1}{2}\lambda\boldsymbol{w}^T\boldsymbol{w}$$

Hessian

$$\boldsymbol{H} = \bar{\boldsymbol{X}}\boldsymbol{R}\bar{\boldsymbol{X}}^T + \lambda\boldsymbol{I}$$

where $R_{nn} = \sigma(\boldsymbol{w}^T\bar{\boldsymbol{x}}_n)(1-\sigma(\boldsymbol{w}^T\bar{\boldsymbol{x}}_n)$

the term λI ensures that $\boldsymbol{H}$ is not singular (eigenvalues $\geq \lambda$)

Prevent against singularity
- If all values are zero
- At least diagonal values are set to lambda
- Eigen values will be nonzero
- It is possible to compute inverse of H

Regularization

- Prevents exploding weights
- Improves generalization
- Ensures numerical stability
- Controls variance

Multiclass Logistic Regression (Softmax)

- For K classes: Softmax $$P(y = k|x) = \frac{e^{w_k^T x}}{\sum_{j=1}^{K} e^{w_j^T x}}$$
    - One weight vector per class
    - Generalization of sigmoid
- Widely used in:
    - Image classification, NLP, recommender systems

Logistic Regression for App recommendation

- Flexibility: millions of features (binary & numerical)

Assume we trained a logistic regression, and would like to do classification
Binary: download / not download app: sigmoid, single vector w for entire model
Multi-class: which app is most likely to download: softmax, vector w for each class k,
same denominator for each class, so maximize just for numerator

- Efficiency: classification by dot products

Multiple classes:

$$c^* = argmax_k \frac{\exp(\boldsymbol{w}_k^T \overline{\boldsymbol{x}})}{\sum_{k'} \exp(w_{k'}^T \overline{\boldsymbol{x}})}$$

$$= argmax_k \, \boldsymbol{w}_k^T \overline{\boldsymbol{x}}$$

Two classes:

$$c^* = \begin{cases} 1 & \sigma(\boldsymbol{w}^T \overline{\boldsymbol{x}}) \geq 0.5 \\ 0 & \text{otherwise.} \end{cases}$$

$$c^* = \begin{cases} 1 & \boldsymbol{w}^T \overline{\boldsymbol{x}} \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Implementation tricks to make it more efficient:
— **exploit sparsity**: multiply only nonzero entries when taking dot product
— **parallelize computation**: GPU: element-wise products in one operation (esp. for multiple classes)

Summary

- Logistic Regression is:
  - Linear classifier
  - Probabilistic model
  - Convex optimization problem
  - One-layer neural network
- Foundation for:
  - Deep learning
  - Cross-entropy loss
  - Backpropagation

Logistic Regression Applications

- Used in:
  - Healthcare risk prediction
  - Credit scoring
  - Ad click prediction
  - Recommender systems
  - Spam detection
- Fast:
  - Just a dot product + sigmoid

Lecture Objectives

- We learned: We learned:
  - How to model $P(y|x)$
  - Why sigmoid is used
  - Maximum likelihood estimation
  - Cross-entropy loss
  - Convex optimization
  - Gradient and Newton methods
  - Regularization
  - Softmax extension

Discriminative vs. Generative

- Logistic Regression:
  - Discriminative model
  - Directly models P(y|x)
- Future Topic:
  - Generative models
  - Model P(x|y)
  - Gaussian class-conditionals
  - Mixture models